

Problem 1

```
In [42]: import qutip as qt
from qutip import Qobj
import numpy as np

# Define polarization states
H = Qobj([[1], [0]]) # Horizontal polarization
A = (1 / np.sqrt(2)) * Qobj([[1], [-1]]) # Anti-diagonal polarization
V = Qobj([[0], [1]]) # Vertical polarization
D = (1 / np.sqrt(2)) * Qobj([[1], [1]]) # Diagonal polarization

num_send = 10000 # the number of photons Alice send
```

```
In [43]: def ADC(qobj):
    """
    translate qobj to char variable, so that the public channel function is

    Parameter(s):
        qobj: the qobj needed to be translated

    Return(s):
        the corresponding character to the input.
    """
    if qobj==H:
        return "H"
    if qobj==V:
        return "V"
    if qobj==D:
        return "D"
    if qobj==A:
        return "A"
```

part a).

The following box belongs to Alice

```
In [44]: def Alice(num_photon):
    """
    "Alice" is a function that accepts an integer num_photon which is the number of photon
    Parameter(s):
        num_photon: the number of photon
    Return(s):
        a list that has two columns, the first column is the index(starting from 0)
    """
    p = 0.2
    return [(i+1, np.random.choice([H, A]), np.random.choice([1, 2], p=[1-p, p])) for i in range(num_photon)]
```

part b).

The following box belongs to Bob

```
In [45]: def Bob(photon_list):
    """
    "Bob" accepts the output of "Alice" and returns only the events where he gets a pin
    Parameter(s):
        photon_list: the output of Alice, a list that consist of many tuple
    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and their index
    """
    received_message=[]
    for index, photon, _ in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V', 'D'])
        if receiver == 'V' and state == 'H' or receiver == 'D' and state == 'A':
            pass # No pin Possible
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', 'Null'], p=[0.1, 0.9]) # simulate
            if pin != 'Null':
                received_message.append((index, pin)) # register the pin
        elif receiver == 'V' and state == 'A': # similar to the last elif
            pin = np.random.choice(['A', 'Null'], p=[0.1, 0.9])
            if pin != 'Null':
                received_message.append((index, pin))
    return received_message
```

```
In [46]: def Bob_1(photon_list):
    """
    "Bob" accepts the output of "Alice" and returns only the events where he gets a pin
    Parameter(s):
        photon_list: the output of Alice, a list that consist of many tuple
    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and their index
```

```

...
received_message=[]
for index, photon, _ in photon_list:
    state = 'A' if photon == A else 'H'
    receiver = np.random.choice(['V','D'])
    if receiver == 'V' and state == 'H' or receiver == 'D' and state == 'A':
        pass # No pin Possible
    elif receiver == 'D' and state == 'H':
        pin = np.random.choice(['H', 'Null']) # simulate the process of
        if pin != 'Null':
            received_message.append((index, pin)) # register the pin
    elif receiver == 'V' and state == 'A': # similar to the last elif
        pin = np.random.choice(['A', 'Null'])
        if pin != 'Null':
            received_message.append((index, pin))
return received_message

```

part c). is two boxes down

part d).

```

In [47]: def Eve(photon_list):
...
    a spy "Eve" who takes the output that "Alice" produces, performs the same
    Parameter(s):
        photon_list: the output of Eve, a list that consist of many tuple of
    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
...
received_message=[]
for index, photon, num_photon in photon_list:
    state = 'A' if photon == A else 'H'
    receiver = np.random.choice(['V','D'])
    if num_photon != 1:
        if receiver == 'V' and state == 'H' or receiver == 'D' and state == 'A':
            received_message.append((index, receiver, 2)) # indicate the
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', receiver]) # simulate the process of
            received_message.append((index, pin, 2))
        elif receiver == 'V' and state == 'A': # similar to the last elif
            pin = np.random.choice(['A', receiver])
            received_message.append((index, pin, 2))
    else:
        # received_message.append((index, state, 1))
        pass

    forged_message = []

    for index, click, num_p in received_message:
        if num_p != 1:
            if click == "V":
                forged_message.append((index, H, num_p))

```

```

        elif click == "D":
            forged_message.append((index, A, num_p))
        elif click == "H":
            forged_message.append((index, H, num_p))
        elif click == "A":
            forged_message.append((index, A, num_p))
    else:
        # forged_message.append((index, click, num_p))
        pass

    return forged_message

```

```

In [48]: def public_channel(click_received, photon_sent):
        """
        This channel is used to varify the click Bob got with the photon Alice s
        Parameter(s):
            click_received: a part of the click bob received

        Return(s):
            check_list: a list that contains for each click Bob claim he receive
            true_rate: the proportion of clicks that Bob got it right.
        """
        check_list = []
        for i, t in enumerate(click_received) :
            index, click = t
            if ADC(photon_sent[index - 1][1]) == click:
                check_list.append((index, True))
            else:
                check_list.append((index, False))
        # calculate the ratio of correct count to the entire compared clicks
        true_count = 0
        for i, b in check_list:
            if b:
                true_count += 1
        true_rate = true_count/len(check_list)
        error_rate = 1-true_rate
        return check_list, error_rate

```

part c).

```

In [49]: photon_sent = Alice(num_send) # the photons Alice sent
        click_received = Bob(photon_sent[0:int(len(photon_sent)/2)]) # the click Bo
        temp, rate = public_channel(click_received, photon_sent)
        print("Error rate: ", rate)

```

Error rate: 0.0

part e).

the sample number 100 is too small to give a result that reflects Eve's smartness/dumbness, statistics are shown in part f).

```
In [50]: photon_sent = Alice(num_send) # the photons Alice sent
         photon_sent_E = Eve(photon_sent) # Big brother(big sister in this case) is
         click_received = Bob_1(photon_sent_E[0:int(len(photon_sent_E)/2)]) # the cl
         temp, rate = public_channel(click_received, photon_sent)
         print('Error rate: ', rate)
```

Error rate: 0.25619834710743805

part f).

```
In [51]: rate_list = []
         num_cr = 0
         repeat = 50
         for i in range(repeat):
             photon_sent = Alice(num_send) # the photons Alice sent
             photon_sent_E = Eve(photon_sent)
             click_received = Bob_1(photon_sent_E) # the click Bob got
             num_cr += len(click_received)
             temp, rate = public_channel(click_received, photon_sent)
             rate_list.append(rate)
         num_cr = num_cr/repeat
         print('Error rate: ', np.mean(rate_list), f'click/photon sent:{num_cr/num_ser
```

Error rate: 0.24180593094372305 click/photon sent:0.0495